



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	B.E.	Semester:	VIII
Course Code:		Course Name:	RL

Name of Student:	BART ANSHU VINOD
Roll No. :	61
Assignment No.:	2
Title of Assignment:	
Date of Submission:	
Date of Correction:	

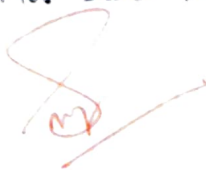
Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Completeness	5	04
Demonstrated Knowledge	3	02
Legibility	2	02
Total	10	08

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Completeness	5	3-4	1-2
Demonstrated Knowledge	3	2	1
Legibility	2	1	0

Checked by

Name of Faculty : Ms. Sneha Yashwanth

Signature : 

Date : 

2. 1) In a racing game AI only knows its current (x, y) position but forget its current speed, why does this breaks the markov property and make it impossible to predict the next state? identify the state action and a reward signal you would use to teach this AI to finish the race quickly without hitting walls.

→ Markov property definition -

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_t, S_{t-1}, S_{t-2}, \dots)$$

- In the racing game -

$$\text{state} = (x, y)$$

- But does not know speed or velocity, then two cars can be at the same position (x, y) but, one moving at 200 km/h, one moving at 20 km/h. If both press "accelerate", fast car may crash into wall, slow car moves safely forward so,

$$P(S_{t+1} | x, y)$$

- why this makes prediction impossible -

$$x_{t+1} = x_t + v \cdot \Delta t$$

Next position depends on velocity, if velocity is unknown, you cannot compute next state, transition probability is ambiguous. Thus here the environment becomes a POMDP (partially observable MDP) instead of MDP.

- Proper state representation -

To satisfy markov property, state should include

$$S = (x, y, v_x, v_y)$$

$$\text{or } S = (x, y, \text{speed}, \text{direction})$$

Optionally add: - distance to wall

- track curvature ahead

2) State the advantages and disadvantages of action-value methods compared to other reinforcement learning techniques with an appropriate example.

→ Action-value methods learn the function:

$$Q(s, a) = E(R_t + \gamma(R_{t+1} + \dots + \gamma^k R_{t+k+1} + \gamma^{k+1} V_{t+k+1}(s, A_{t+k+1} = a))$$

Instead of directly learning the policy, they learn Q-values and then derive the policy:

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

- Eg. grid world game -

Agent must reach goal (+10 reward). each step cost -1. falling in trap gives -50. An action-value method learns $Q(s, up)$,

$Q(s, down)$, $Q(s, left)$, $Q(s, right)$, choose action with highest Q-value.

- Advantages of action-value methods -

No need to know transition probability, works directly from interaction

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

No separate policy representation required. Eg. chess moves, grid navigation, Atari games, can learn optimal policy even when exploring randomly. This improves sample efficiency.

(i) Poor for continuous action spaces

(ii) overestimation bias

(iii) scalability issues

(iv) indirectly optimization of policy

(v)

Eg. "Racing game", steering angle is continuous (-30° to $+30^\circ$)
action value method - must discretize steering

- loses precision gradient